Assignment 7: Deploy a simple Web Application on EC2 server using AWS Database services

Overview

In this assignment, you will take the role of a AWS database services. Your mission is to design, implement, the database and display the database record through AWS EC2 web server page on browser. The focus is on creating a highly available, scalable, and secure cloud environment using AWS managed services. This deployment includes real-world cloud engineering tasks involving Amazon RDS, and Redis related AWS networking components. You will be responsible for provisioning, deploying, and verifying both the backend and frontend applications using Nodejs.

Deliverables

This is a group assignment. All task should be implemented by student group or it can be implemented individually (**Access AWS academy: Sandbox → Sandbox Environment**). Each student must submit complete presentation slides or PDF.

1. **Presentation making guidelines**
   - Use professional formatting (legible fonts, consistent colors, organized layout).
   - Ensure clarity: avoid overcrowding slides with text; use bullet points and visuals.
   - Title slide with student name(s), course, and assignment title
   - Provide output screenshots as evidence of work completed..
   - Be prepared to answer questions about the process and challenges.
2. **Task complementation**: Each AWS task is evaluated based on correctness, completeness, and evidence (e.g., screenshots of output with brief description). **(60 points total)**.
3. **Presentation Quality (20 points total)**
   - Clear structure (intro, results, discussion, conclusion)
   - Slides visually organized and professional, for example, adding appropriate title of each task, pictures, transitions, animations Screenshots, included as evidence of each task results with proper description of each task results.
4. **Demonstration (in-class or video recorded) (10 point total)**
   - A 9–12 minute walk-through of the slides and discussion of your results.
5. **Professionalism & Delivery (10 points total)**

- Confident and clear presentation delivery
- Each student submit slides (PowerPoint or PDF) to Moodle by the deadline.

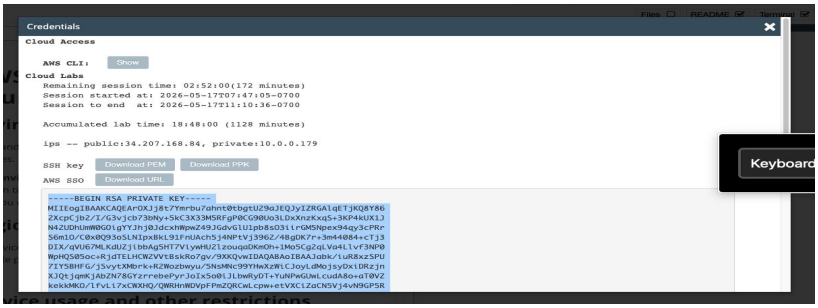
Task 1: Create an public EC2 Ubuntu user instance

- **Instructions:**

1. Navigate to the **EC2** service.
2. Click on Launch Instance
3. **Launch EC2 (Public Instance):**
    - **Name: studentDB**
    - **Application and OS Images (AMI):** Select Ubuntu (Free tier eligible).
    - **Instance type:** Select **t2.micro** (Free tier eligible).
    - **Key pair (login):** select an existing key pair.
    - **Network settings:** Click **Edit**.
        - **VPC:** Select deafult **VPC**.
        - **Subnet:** Select any existing **public-subnet of US-east-1a**
        - **Auto-assign public IP:** Select **Enable**.
    - **Firewall (security groups):**
        - Select **Create a new security group**.
        - **Security group name:** `public-sg`
        - **Description:** Allows SSH and ICMP
        - **Inbound rules:**
            - Rule 1: Type **SSH**, Source **Custom** use `0.0.0.0/0`. Port 22
            - Rule 2: Click **Add security group rule**. Type **Custom TCP**, Source **Custom** `0.0.0.0/0. port: 3000`
        - Click **Launch instance**
4. Now Navigate to the Cloud Foundations – Sandbox AWS academy window
    - Click on details then Show



- Copy the whole key to access public EC2 through cloudshell.



- **Open Cloudshell terminal**
  - Create key file for access EC2 instance through SSH
    - nano studentkey.pem
    - Just paste the key which you have copied from AWS academy sandbox environment window.
    - Save the studentkey.pem file by Ctrl+Z and save it.
    - Give the permission to key file
      - chmod 400 studentkey.pem
    - login to ec2 instance through SSH
      - ssh -i studentkey.pem ubuntu@PUBLIC-IP

5. Update ubuntu
   - sudo apt update -y
6. Install MySQL server
   - sudo apt install mysql-server -y
     - Check version
       - mysql --version

Task 2: Create MySQL database accessible from anywhere.

- ☑ Open **AWS Console → Aurora and RDS → Databases → Create Database**
- ☑ Select Engine type as **MySQL**
- ☑ Choose a database creation method select → **Full configuration**
- ☑ Templates → choose Dev/test
- ☑ Availability and durability → Deployment options → Single-AZ DB instance deployment (1 instance)
- ☑ Settings
  - Default MySQL community, Version leave default
  - In DB instance identifier set name as **student db**
  - Credentials Settings
    - master username set as **admin**
    - Credentials management: select **self managed**
      - Make password: admin123
    - Additional credential settings: select **password authentication**
- ☑ Instance configuration
  - Select **Burstable classes (includes t classes)**
  - Instance type: Select **db.t3.micro**
- ☑ **Storage configure**
  - Storage type: General Purpose (SSD) gp3
  - Allocated storage: 20 GiB
  - Other setting leave as default.
- ☑ **Connectivity:**
  - **Compute resources**
    - Select Don't connect to an EC2 compute resource
    - VPC: default
    - subnet group leave as default
    - Public access: Yes
    - Leave other settings as defaults.
    - Availability Zone
      - select US-east-1a
    - RDS proxy leave as default
- ☑ Monitoring
  - select: Database Insights - Standard
  - Under **Monitoring** expand **Additional configuration**.
    - Uncheck **Enable Enhanced monitoring**.
  - Under **Additional configuration**, configure
    - **Initial database name:** studentlab

- DB parameter group leave as default
- Uncheck **Enable automatic backups**.
- Uncheck **Enable encryption**

☑ Click **Create** and wait until the status is *Available*.

☑ **Save for later:**
- Endpoint
- Port: 3306
- DB name: studentlab
- User: admin
- Password: admin123

You'll create the schema later via **RDS Query Editor v2**.

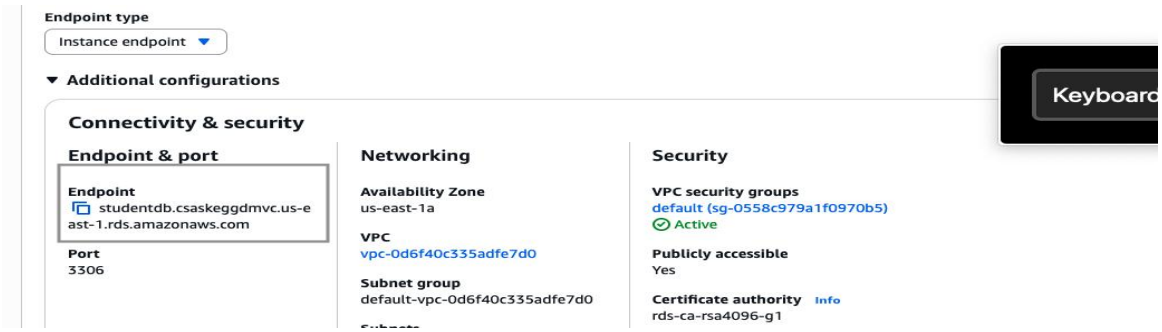
Task 3: Connect MySQL database through EC2 instances.

☑ Go back to cloudshell.

☑ Access to EC2 Ubuntu instance through SSH

☑ Before connecting to database through EC2 set the Inbound security rule of Database.

- Go to database.
- Select created studentdb
- Scroll down and in the Security group rules section
  - click on security of type EC2 Security Group - Inbound
    - Scroll down and click on Tab Inbound security rules
      - Then click on Edit inbound rules
      - Click Add security group rule. Type MYSQL/Aurora, Source Custom 0.0.0.0/0. port: 3306
      - Save the security rules.
- Copy your database endpoint



- Now connect to database using command
  - mysql -h paste your RDS endpoint address -P 3306 -u admin -p
  - Enter your database password
  - Now you connected to MySQL database.

Task 4: Create Database, Table and insert record in the table.

☑ Create Database inside MySQL using command
- CREATE DATABASE database name;

☑ Use database using command
- USE database name;

☑ Create Table using command

```
CREATE TABLE students (
id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(50),
course VARCHAR(50)
);
```

☑ Insert record in to the table

```
INSERT INTO students(name, course)
VALUES
('John', 'AWS'),
('Alice', 'Cloud Computing'),
('Bob', 'DevOps');
```

☑ **Verify the inserted data**

SELECT * FROM students; (if table name is students)

☑ After creating table and inserting data exit from SQL using

EXIT;

Task 5: Display the student record on web page through EC2 web server

1. Install NodeJS on EC2
   - sudo apt update
   - sudo apt install nodejs npm -y
2. Create web project folder using command
   - mkdir webapp
   - cd webapp
3. Initialize project
   - npm init -y
4. Install npm packages and create app.js file
   - npm install express mysql2
   - create app.js using nano app.js
   - paste the following code in the app.js file.

```
const express = require('express');
const mysql = require('mysql2');

const app = express();
const db = mysql.createConnection({
   host: 'paste your database endpoint url',
   user: 'admin',
   password: 'database password',
   database: 'database name'
});

db.connect((err) => {
   if(err){
      console.log(err);
   } else {
      console.log('RDS Connected');
   }
});

app.get('/', (req, res) => {

   db.query('SELECT * FROM students', (err, results) => {
```

```
        if(err){
            res.send(err);
        } else {

            let html = `
            <html>
            <head>
                <title>Student Records</title>
            </head>

            <body>

                <h1>Student Records</h1>

                <table border="1" cellpadding="10">

                    <tr>
                        <th>ID</th>
                        <th>Name</th>
                        <th>Course</th>
                    </tr>
            `;
            results.forEach(student => {
                html += `
                    <tr>
                        <td>${student.id}</td>
                        <td>${student.name}</td>
                        <td>${student.course}</td>
                    </tr>
                `;
            });
            html += `
                </table>
            </body>
            </html>
            `;
            res.send(html);
        }
    });
});
app.listen(3000, () => {
```

```
    console.log('Server running on port 3000');
});
```

5. save the app.js file by using CTRL+Z, Confirm Y (Yes) and press enter
6. Run you r app.js
   - node app.js
   - It will display: RDS Connected
     Server running on port 3000
7. Now open browser and type URL
   http://EC2-PUBLIC-IP:3000

8. You should now see:
   - Student Records webpage
   - Data loaded from RDS database

Task 6: Deploying a Redis Cache-Based Web Application on AWS EC2

1. If EC2 instance not launched then launched EC2 instance using the steps of Task 1 from steps 1 to step 5.
   a) Then install Redis using command: **sudo apt install redis-server -y**
   b) Start Redis using command: **sudo systemctl start redis-server**
   c) Enable auto-start: **sudo systemctl enable redis-server**
   d) Check status: **sudo systemctl status redis-server**
2. Test Redis.
   a) using command: **redis-cli**
   b) Store data in to the redis: **SET course AWS** (This command is used for storing string type data).
   c) For more way of storing data in Redis follow the below link document
      - https://redis.io/docs/latest/commands/redis-8-6-commands/#set-commands

   d) Now retrieve the data using command: GET course (course is the key)
   e) Exit from redis cli using **exit** command
3. Install node JS if ot install in EC2 instance server.
   a) sudo apt install nodejs npm -y
   b) Create web application folder using command:
      i. **mkdir redisapp**
      ii. then navigate to folder using command: **cd redisapp**
   c) Initialize project: **npm init -y**
   d) Install packages: **npm install express redis**
   e) Create app.js file using command: **nano app.js**
   f) Copy and paste the below code in to the app.js file.

```
const express = require('express');
const redis = require('redis');

const app = express();
```

```
const client = redis.createClient();

(async () => {

    await client.connect();

    console.log('Redis Connected');

    await client.set('message', 'Welcome to Redis Cache on AWS EC2');

})();

app.get('/', async (req, res) => {

    const value = await client.get('message');

    res.send(`
        <html>
        <head>
            <title>Redis Demo</title>
        </head>

        <body>

            <h1>Redis Cache Demo</h1>

            <p>${value}</p>

        </body>
        </html>
    `);
});

app.listen(3000, () => {

    console.log('Server running on port 3000');

});
```

g) Save app.js file and run the application using command: **node app.js**
h) Now open browser and type URL
    i. http://EC2-PUBLIC-IP:3000
i) Just see the redis data results in the browser.

Task 7: Reflections Questions

- After implementing the web applications using Amazon RDS, and Redis on Amazon EC2, compare the differences between relational databases (RDS MySQL), and in-memory cache databases (Redis) in terms of data structure, storage method, performance, scalability, and real-world use cases.

----------END----------

Dr. Rohit Kumar Kasera

Lecturer